School of Future Tech

Case Study Report

ICU Resource Management System

by

- Student Name: Nandeni Tiwari
- Roll No: 150096725033
- Division: Jeff Bezos
- Subject: C++
- Academic Year: 2025-29

1. Introduction to the Case Study

Hospitals often face challenges in managing Intensive Care Unit (ICU) resources efficiently. ICU beds, medical equipment such as ventilators, and healthcare staff are limited resources that must be allocated properly to ensure critical patients receive timely treatment. During emergencies or periods of high patient inflow, hospitals must quickly decide how to distribute beds, equipment, and medical staff while maintaining accurate records of admissions and discharges.

The ICU Resource Management System is designed to help hospitals monitor and manage ICU resources more effectively. This system keeps track of ICU beds, available medical equipment, staff allocation, and patient admissions. It also provides reports on resource utilization and alerts when resources reach critical levels.

The system is implemented using C++ and follows an object-oriented programming approach. The application is menu-driven, allowing hospital administrators to easily interact with the system to perform operations such as registering patients, assigning ICU beds, monitoring equipment usage, generating reports, and maintaining logs of hospital activities.

2. Problem Statement / Case Background (Abstract)

In many hospitals, ICU resources are limited and must be managed carefully. Manual tracking of ICU beds, ventilators, and staff can lead to delays, mismanagement, or incorrect allocation of resources. Without proper monitoring, hospitals may face issues such as ICU bed shortages, equipment unavailability, and staff overload.

The objective of this case study is to develop a simple ICU Resource Management System that can track ICU beds, equipment availability, patient admissions, and staff assignments. The system also provides real-time alerts when resources reach critical thresholds and generates reports to analyze ICU utilization.

This system helps hospital administrators make better decisions regarding resource allocation and improves the overall efficiency of ICU management.

3. Problem Statement / Case Study Design

The ICU Resource Management System is designed as a menu-driven application where different modules handle specific hospital operations.

The system includes the following modules:

ICU Bed Management

This module keeps track of the total ICU bed capacity in the hospital. Each bed has a unique ID and a status such as available, occupied, reserved, or under maintenance. The system displays the current status of all ICU beds so that administrators can monitor bed availability.

Equipment Tracking

Medical equipment such as ventilators are essential for ICU patients. The system tracks the number of available, in-use, and faulty equipment. It also allows equipment to be marked as faulty or repaired so that maintenance activities can be monitored.

Patient Assignment

Patients can be registered in the system by entering details such as patient ID, name, diagnosis, and severity level. Based on bed availability, the system automatically assigns an ICU bed and equipment to the patient. If beds are not available, the patient is placed in a waiting list, and the system prioritizes patients based on severity level.

Staff Allocation

The system tracks the number of doctors and nurses available in the ICU. When a patient is admitted, the system assigns a doctor and nurse automatically. It also monitors staff workload and generates warnings if the number of patients exceeds available staff.

Reporting System

The system generates reports that provide useful statistics such as ICU bed occupancy rate, equipment utilization, and staff workload. These reports help administrators understand how ICU resources are being used.

Alert System

Alerts are generated when important conditions occur, such as ICU occupancy exceeding a certain threshold, equipment availability dropping below a safe level, or staff becoming overloaded.

4. Methods & Algorithms Technology Applied in the Case Study

The system is implemented using the C++ programming language and follows the Object-Oriented Programming (OOP) paradigm. Different classes are used to represent components of the ICU system.

Classes Used

The main classes implemented in the system are:

- **Patient** – stores patient details such as ID, name, diagnosis, severity level, and assigned bed.
- **ICUBed** – stores ICU bed information including bed ID and current status.
- **Equipment** – tracks available, in-use, and faulty equipment.
- **Staff** – stores the number of doctors and nurses available.
- **FileManager** – handles file operations such as logging patient admissions and discharges.
- **ReportGenerator** – generates reports and exports them in CSV format.
- **ResourceManager** – controls the overall system and manages beds, patients, equipment, and staff.

Data Structures Used

Vectors are used to store collections of patients and ICU beds dynamically. A waiting list is also implemented using a vector to store patients when ICU beds are not available.

Algorithms Used

Bed Allocation Algorithm

The system checks all ICU beds and assigns the first available bed to the patient. If no beds are available, the patient is placed in a waiting list.

Severity-Based Priority Allocation

Patients in the waiting list are sorted based on severity level. When a bed becomes available, the patient with the highest severity level is assigned the bed first.

Resource Monitoring Algorithm

The system calculates metrics such as bed occupancy rate and equipment utilization percentage using simple mathematical formulas.

Staff Workload Monitoring

The system compares the number of admitted patients with available staff to determine whether staff are overloaded.

5. Case Study Implementation Details and Snapshots

The system is implemented as a console-based application in C++.

Key Functionalities Implemented

1. ICU Bed Dashboard

The system displays the status of all ICU beds, showing whether they are available, occupied, reserved, or under maintenance.

2. Patient Registration

Administrators can register a new patient by entering patient ID, name, diagnosis, and severity level.

3. Automatic Bed Assignment

When a patient is registered, the system automatically assigns an available ICU bed and equipment if available.

4. Waiting List System

If all ICU beds are occupied, the patient is added to a waiting list and prioritized based on severity.

5. Equipment Monitoring

The system tracks equipment availability and allows equipment to be marked faulty or repaired.

6. Staff Assignment

Doctors and nurses are automatically assigned to patients when they are admitted.

7. Discharge System

When a patient is discharged, the bed and equipment are released and made available for other patients.

8. Report Generation

The system generates ICU utilization reports and exports them to a CSV file.

9. Logging System

Patient admissions and discharges are recorded in log files for future reference.

Snapshots

Snapshots include:

- Main code

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <limits>
#include <ctime>
using namespace std;

static string nowTime()
{
    time_t t = time(nullptr);
    tm *lt = localtime(&t);
    char buf[64];
    strftime(buf, sizeof(buf), "%Y-%m-%d %H:%M:%S", lt);
    return string(buf);
```

```
}

static void clearLine()
{
    cin.ignore(numeric_limits<streamsize>::max(), '\n');
}


class Patient
{
public:
    int id = 0;
    string name;
    string diagnosis;
    int severity = 1; // 1-5
    int bedID = -1;


    int doctorAssigned = -1;
    int nurseAssigned = -1;


    bool equipmentAssigned = false;
};


class ICUBed
{
public:
    int id = 0;
    string status;
};
```

```
class Equipment
{
public:
    int available = 0;
    int inUse = 0;
    int faulty = 0;
};


class Staff
{
public:
    int doctors = 0;
    int nurses = 0;
};


class FileManager
{
public:
    static void logAdmission(const Patient &p)
    {
        ofstream file("admissions.txt", ios::app);
        if (!file)
            return;

        file << nowTime()
             << " | Patient " << p.id
             << " admitted | Bed " << p.bedID
             << " | Sev " << p.severity
             << " | Dr " << p.doctorAssigned
             << " | Nurse " << p.nurseAssigned
```

```
            << " | Equip " << (p.equipmentAssigned ? "YES" : "NO")
            << " | Dx: " << p.diagnosis
            << "\n";
    }

    static void logDischarge(int id)
    {
        ofstream file("discharge.txt", ios::app);
        if (!file)
            return;

        file << nowTime() << " | Patient " << id << " discharged\n";
    }

    static void ensureReportHeader()
    {
        ifstream in("report.csv");
        bool empty = true;
        if (in)
            empty = (in.peek() == ifstream::traits_type::eof());
        in.close();

        if (empty)
        {
            ofstream out("report.csv", ios::app);
            if (!out)
                return;
            out <<
"Timestamp,TotalBeds,OccupiedBeds,AvailableEquip,InUseEquip,FaultyEquip,TotalPa
tients,BedOccupancyRate,EquipUtilRate\n";
        }
    }
```

```
    static void exportReportRow(const string &row)
    {
        ensureReportHeader();
        ofstream file("report.csv", ios::app);
        if (!file)
            return;
        file << row << "\n";
    }

    static void viewLogs(const string &filename)
    {
        ifstream file(filename);
        if (!file)
        {
            cout << "\nCould not open " << filename << " (file not found
yet).\n";
            return;
        }

        string line;
        cout << "\n---- " << filename << " ----\n";
        while (getline(file, line))
            cout << line << "\n";
    }
};

class ReportGenerator
{
public:
    static void generate(int bedsTotal, int occupied,
                         const Equipment &eq, const Staff &staff,
                         int totalPatients)
    {
```

```
        cout << "\n---- ICU UTILIZATION REPORT ----\n";
        cout << "Total Beds: " << bedsTotal << "\n";
        cout << "Occupied Beds: " << occupied << "\n";
        cout << "Available Beds: " << (bedsTotal - occupied) << "\n";

        double bedRate = 0;
        if (bedsTotal > 0)
            bedRate = (double)occupied / bedsTotal * 100.0;
        cout << "Bed Occupancy Rate: " << bedRate << "%\n";

        int usableEquip = eq.available + eq.inUse; // exclude faulty
        double equipUtil = 0;
        if (usableEquip > 0)
            equipUtil = (double)eq.inUse / usableEquip * 100.0;

        cout << "\nEquipment\n";
        cout << "Available: " << eq.available << "\n";
        cout << "In Use: " << eq.inUse << "\n";
        cout << "Faulty: " << eq.faulty << "\n";
        cout << "Equipment Utilization (of usable): " << equipUtil << "%\n";

        cout << "\n--- Staff Workload ---\n";
        cout << "Doctors on duty: " << staff.doctors << "\n";
        cout << "Nurses on duty: " << staff.nurses << "\n";

        if (staff.nurses > 0)
            cout << "Patients per Nurse: " << (double)totalPatients /
    staff.nurses << "\n";
        if (staff.doctors > 0)
            cout << "Patients per Doctor: " << (double)totalPatients /
    staff.doctors << "\n";

        FileManager::exportReportRow(
```

```
            nowTime() + "," +
            to_string(bedsTotal) + "," +
            to_string(occupied) + "," +
            to_string(eq.available) + "," +
            to_string(eq.inUse) + "," +
            to_string(eq.faulty) + "," +
            to_string(totalPatients) + "," +
            to_string(bedRate) + "," +
            to_string(equipUtil));
    }
};


class ResourceManager
{
    vector<Patient> patients;
    vector<Patient> waitingList;
    vector<ICUBed> beds;

    Equipment equipment;
    Staff staff;

    int nextDoctor = 1;
    int nextNurse = 1;

public:
    ResourceManager()
    {
        int capacity;
        cout << "Enter ICU Bed Capacity: ";
        cin >> capacity;

        for (int i = 1; i <= capacity; i++)
```

```
        {
            ICUBed b;
            b.id = i;
            b.status = "available";
            beds.push_back(b);
        }

        equipment.available = 3;
        equipment.inUse = 0;
        equipment.faulty = 0;

        staff.doctors = 2;
        staff.nurses = 4;
    }



    void showBeds()
    {
        cout << "\nICU BED STATUS\n";
        for (auto b : beds)
            cout << "Bed " << b.id << " : " << b.status << "\n";
    }



    void changeBedStatus()
    {
        int id;
        string status;

        cout << "Enter Bed ID: ";
        cin >> id;

        cout << "Enter status (available/occupied/reserved/maintenance): ";
```

```
        cin >> status;

        if (!(status == "available" || status == "occupied" || status ==
"reserved" || status == "maintenance"))
        {
            cout << "Invalid status.\n";
            return;
        }

        for (auto &b : beds)
        {
            if (b.id == id)
            {
                b.status = status;
                cout << "Status updated.\n";
                return;
            }
        }
        cout << "Bed not found.\n";
    }


    void addPatient()
    {
        Patient p;

        cout << "Enter Patient ID: ";
        cin >> p.id;
        clearLine();

        cout << "Enter Name: ";
        getline(cin, p.name);
```

```
        cout << "Enter Diagnosis: ";
        getline(cin, p.diagnosis);

        cout << "Enter Severity (1-5): ";
        cin >> p.severity;

        if (p.severity < 1)
            p.severity = 1;
        if (p.severity > 5)
            p.severity = 5;

        if (p.severity >= 4)
            cout << "High Priority Patient\n";

        assignBedOrWait(p);
    }



    void assignStaff(Patient &p)
    {
        if (staff.doctors > 0)
        {
            p.doctorAssigned = nextDoctor;
            nextDoctor++;
            if (nextDoctor > staff.doctors)
                nextDoctor = 1;
        }

        if (staff.nurses > 0)
        {
            p.nurseAssigned = nextNurse;
            nextNurse++;
            if (nextNurse > staff.nurses)
```

```
                nextNurse = 1;
        }

        cout << "Doctor assigned: " << (p.doctorAssigned == -1 ? 0 :
p.doctorAssigned) << "\n";
        cout << "Nurse assigned: " << (p.nurseAssigned == -1 ? 0 :
p.nurseAssigned) << "\n";
    }


    void assignBedOrWait(Patient p)
    {

        for (auto &b : beds)
        {
            if (b.status == "available")
            {

                p.bedID = b.id;


                b.status = "occupied";


                assignStaff(p);


                if (equipment.available > 0)
                {
                    equipment.available--;
                    equipment.inUse++;
                    p.equipmentAssigned = true;
                    cout << "Equipment assigned.\n";
```

```
                }
                else
                {
                    p.equipmentAssigned = false;
                    cout << "Warning: No equipment available right now.\n";
                }



                patients.push_back(p);

                cout << "Bed " << b.id << " assigned.\n";



                FileManager::logAdmission(p);
                return;
            }
        }



        waitingList.push_back(p);
        cout << "No ICU Beds Available. Patient added to waiting list.\n";
        cout << "Waiting List Size: " << waitingList.size() << "\n";
    }



    void allocateFromWaiting()
    {
        if (waitingList.empty())
            return;



        int bestIdx = 0;
        for (int i = 1; i < (int)waitingList.size(); i++)
```

```
        {
            if (waitingList[i].severity > waitingList[bestIdx].severity)
                bestIdx = i;
        }

        Patient p = waitingList[bestIdx];
        waitingList.erase(waitingList.begin() + bestIdx);

        cout << "\nAllocating bed to waiting patient ID " << p.id
             << " (Severity " << p.severity << ")\n";

        assignBedOrWait(p);
    }



    void discharge()
    {
        int id;
        cout << "Enter Patient ID: ";
        cin >> id;

        for (int i = 0; i < (int)patients.size(); i++)
        {
            if (patients[i].id == id)
            {
                int bed = patients[i].bedID;



                for (auto &b : beds)
                {
                    if (b.id == bed)
                    {

```

```
                        b.status = "available";
                        break;
                    }
                }

                if (patients[i].equipmentAssigned)
                {
                    if (equipment.inUse > 0)
                        equipment.inUse--;
                    equipment.available++;
                }

                patients.erase(patients.begin() + i);

                FileManager::logDischarge(id);

                cout << "Patient discharged.\n";



                allocateFromWaiting();
                return;
            }
        }
        cout << "Patient not found.\n";
    }



    void showEquipment()
    {
        cout << "\nEquipment Status\n";
        cout << "Available: " << equipment.available << "\n";
        cout << "In Use: " << equipment.inUse << "\n";
        cout << "Faulty: " << equipment.faulty << "\n";
```

```
        if (equipment.available < 2)
            cout << "Alert: Equipment below minimum threshold!\n";
    }



    void markFaulty()
    {
        cout << "\n1. Mark AVAILABLE equipment faulty\n";
        cout << "2. Mark IN-USE equipment faulty (urgent)\n";
        cout << "Enter choice: ";
        int c;
        cin >> c;

        if (c == 1)
        {
            if (equipment.available > 0)
            {
                equipment.available--;
                equipment.faulty++;
                cout << "Equipment marked faulty.\n";
            }
            else
            {
                cout << "No available equipment to mark faulty.\n";
            }
        }
        else if (c == 2)
        {
            if (equipment.inUse > 0)
            {
                equipment.inUse--;
                equipment.faulty++;
```

```
                cout << "In-use equipment marked faulty.\n";
            }
            else
            {
                cout << "No in-use equipment to mark faulty.\n";
            }
        }
        else
        {
            cout << "Invalid.\n";
        }
    }

    /* Repair Faulty Equipment */
    void repairFaulty()
    {
        if (equipment.faulty > 0)
        {
            equipment.faulty--;
            equipment.available++;
            cout << "One faulty equipment repaired and moved to available.\n";
        }
        else
        {
            cout << "No faulty equipment to repair.\n";
        }
    }

     void showStaff()
    {
        cout << "\nStaff Status\n";
        cout << "Doctors: " << staff.doctors << "\n";
        cout << "Nurses: " << staff.nurses << "\n";
```

```
        if ((int)patients.size() > staff.nurses)
            cout << "Warning: Staff overload!\n";
    }



    void showWaitingList()
    {
        cout << "\n---- WAITING LIST ----\n";
        if (waitingList.empty())
        {
            cout << "No patients in waiting list.\n";
            return;
        }

        for (auto &p : waitingList)
            cout << "ID: " << p.id
                 << " | Name: " << p.name
                 << " | Sev: " << p.severity
                 << " | Dx: " << p.diagnosis << "\n";
    }



    void report()
    {
        int occupied = 0;
        for (auto b : beds)
            if (b.status == "occupied")
                occupied++;

        ReportGenerator::generate((int)beds.size(), occupied, equipment, staff,
(int)patients.size());
```

```
        double rate = 0;
        if (!beds.empty())
            rate = (double)occupied / beds.size() * 100.0;

        if (rate >= 80)
            cout << "ALERT: ICU occupancy above threshold (80%)\n";

        if (occupied == (int)beds.size() && !beds.empty())
            cout << "ALERT: ICU FULL\n";
    }



    void viewLogs()
    {
        cout << "\n1. Admission Logs\n";
        cout << "2. Discharge Logs\n";
        cout << "Enter choice: ";
        int c;
        cin >> c;

        if (c == 1)
            FileManager::viewLogs("admissions.txt");
        else if (c == 2)
            FileManager::viewLogs("discharge.txt");
        else
            cout << "Invalid.\n";
    }
};



int main()
{
    ResourceManager icu;
```

```
    int choice;

    while (true)
    {
        cout << "\n===== ICU RESOURCE SYSTEM =====\n";
        cout << "1. Show Beds\n";
        cout << "2. Register Patient\n";
        cout << "3. Discharge Patient\n";
        cout << "4. Equipment Status\n";
        cout << "5. Mark Equipment Faulty\n";
        cout << "6. Repair Faulty Equipment\n";
        cout << "7. Staff Status\n";
        cout << "8. Generate Report\n";
        cout << "9. Change Bed Status\n";
        cout << "10. View Logs\n";
        cout << "11. Show Waiting List\n";
        cout << "12. Exit\n";

        cout << "Enter Choice: ";
        cin >> choice;

        switch (choice)
        {
        case 1:
            icu.showBeds();
            break;
        case 2:
            icu.addPatient();
            break;
        case 3:
            icu.discharge();
            break;
        case 4:
```

```
            icu.showEquipment();
            break;
        case 5:
            icu.markFaulty();
            break;
        case 6:
            icu.repairFaulty();
            break;
        case 7:
            icu.showStaff();
            break;
        case 8:
            icu.report();
            break;
        case 9:
            icu.changeBedStatus();
            break;
        case 10:
            icu.viewLogs();
            break;
        case 11:
            icu.showWaitingList();
            break;
        case 12:
            return 0;
        default:
            cout << "Invalid choice\n";
        }
    }
}
```

- Program execution menu

```
heyy.nandeni@Nandenis-MacBook-Air c++ project % ./icu
Enter ICU Bed Capacity: 4

===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
```

- ICU bed status display

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 1

ICU BED STATUS
Bed 1 : available
Bed 2 : available
Bed 3 : available
Bed 4 : available
```

- Patient registration screen

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 2
Enter Patient ID: 104
Enter Name: marianne
Enter Diagnosis: severe head concussion
Enter Severity (1-5): 5
High Priority Patient
Doctor assigned: 1
Nurse assigned: 1
Equipment assigned.
Bed 1 assigned.
```

- Patient Discharge

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 10

1. Admission Logs
2. Discharge Logs
Enter choice: 2

---- discharge.txt ----
2026-03-05 20:14:26 | Patient 102 discharged
```

- View logs

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 10

1. Admission Logs
2. Discharge Logs
Enter choice: 1

---- admissions.txt ----
2026-03-05 20:11:59 | Patient 101 admitted | Bed 1 | Sev 5 | Dr 1 | Nurse 1 | Equip YES | Dx: appendicitis
2026-03-05 20:12:33 | Patient 102 admitted | Bed 2 | Sev 2 | Dr 2 | Nurse 2 | Equip YES | Dx: mild fever
2026-03-05 23:14:44 | Patient 104 admitted | Bed 1 | Sev 5 | Dr 1 | Nurse 1 | Equip YES | Dx: severe head concus
sion
```

- Equipment monitoring screen

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 4

Equipment Status
Available: 2
In Use: 1
Faulty: 0
```

- Mark faulty equipment

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 5

1. Mark AVAILABLE equipment faulty
2. Mark IN-USE equipment faulty (urgent)
Enter choice: 2
In-use equipment marked faulty.
```

- Faulty equipment fixed

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 6
One faulty equipment repaired and moved to available.
```

- Staff Status Monitoring

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 7

Staff Status
Doctors: 2
Nurses: 4
```

- Generated ICU report

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 8

---- ICU UTILIZATION REPORT ----
Total Beds: 4
Occupied Beds: 1
Available Beds: 3
Bed Occupancy Rate: 25%

Equipment
Available: 3
In Use: 0
Faulty: 0
Equipment Utilization (of usable): 0%

--- Staff Workload ---
Doctors on duty: 2
Nurses on duty: 4
Patients per Nurse: 0.25
Patients per Doctor: 0.5
```

- Patient waitlist

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 11

---- WAITING LIST ----
No patients in waiting list.
```

Patients are waitlisted when there are no beds available and when a bed is available based on the severity and emergency of the patient the bed is given out to the one which is more in need.

- waitlist

```
Enter Choice: 2
Enter Patient ID: 107
Enter Name:
Enter Diagnosis: fracture
Enter Severity (1-5): 5
High Priority Patient
No ICU Beds Available. Patient added to waiting list.
Waiting List Size: 2

===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 11

---- WAITING LIST ----
ID: 107 | Name: lia | Sev: 2 | Dx: cough and cold
ID: 107 | Name:  | Sev: 5 | Dx: fracture
```

- Bed assigned to patient with high severity level

```
Enter Choice: 3
Enter Patient ID: 105
Patient discharged.

Allocating bed to waiting patient ID 107 (Severity 5)
Doctor assigned: 1
Nurse assigned: 3
Equipment assigned.
Bed 1 assigned.
```

- Change bed status

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 9
Enter Bed ID: 101
Enter status (available/occupied/reserved/maintenance): maintenance
Bed not found.
```

- Exit

```
===== ICU RESOURCE SYSTEM =====
1. Show Beds
2. Register Patient
3. Discharge Patient
4. Equipment Status
5. Mark Equipment Faulty
6. Repair Faulty Equipment
7. Staff Status
8. Generate Report
9. Change Bed Status
10. View Logs
11. Show Waiting List
12. Exit
Enter Choice: 12
```

6. Case Study Results and Conclusion

The ICU Resource Management System successfully demonstrates how hospital resources can be managed using a simple software application. The system allows administrators to track ICU beds, manage patient admissions, monitor equipment usage, and assign staff efficiently.

The implementation also provides important features such as priority-based patient allocation, equipment monitoring, and alert generation when resources become limited. Reports generated by the system help administrators analyze resource utilization and improve decision-making.

Overall the system provides a basic but effective solution for ICU resource monitoring and demonstrates the application of object-oriented programming concepts in solving real-world healthcare management problems.

7. References

GeeksforGeeks. (2023). *C++ Programming Language Tutorial.*
https://www.geeksforgeeks.org/c-plus-plus/

Agency for Healthcare Research and Quality (AHRQ). (2022). *Hospital Resource Management and Patient Safety.*
https://www.ahrq.gov